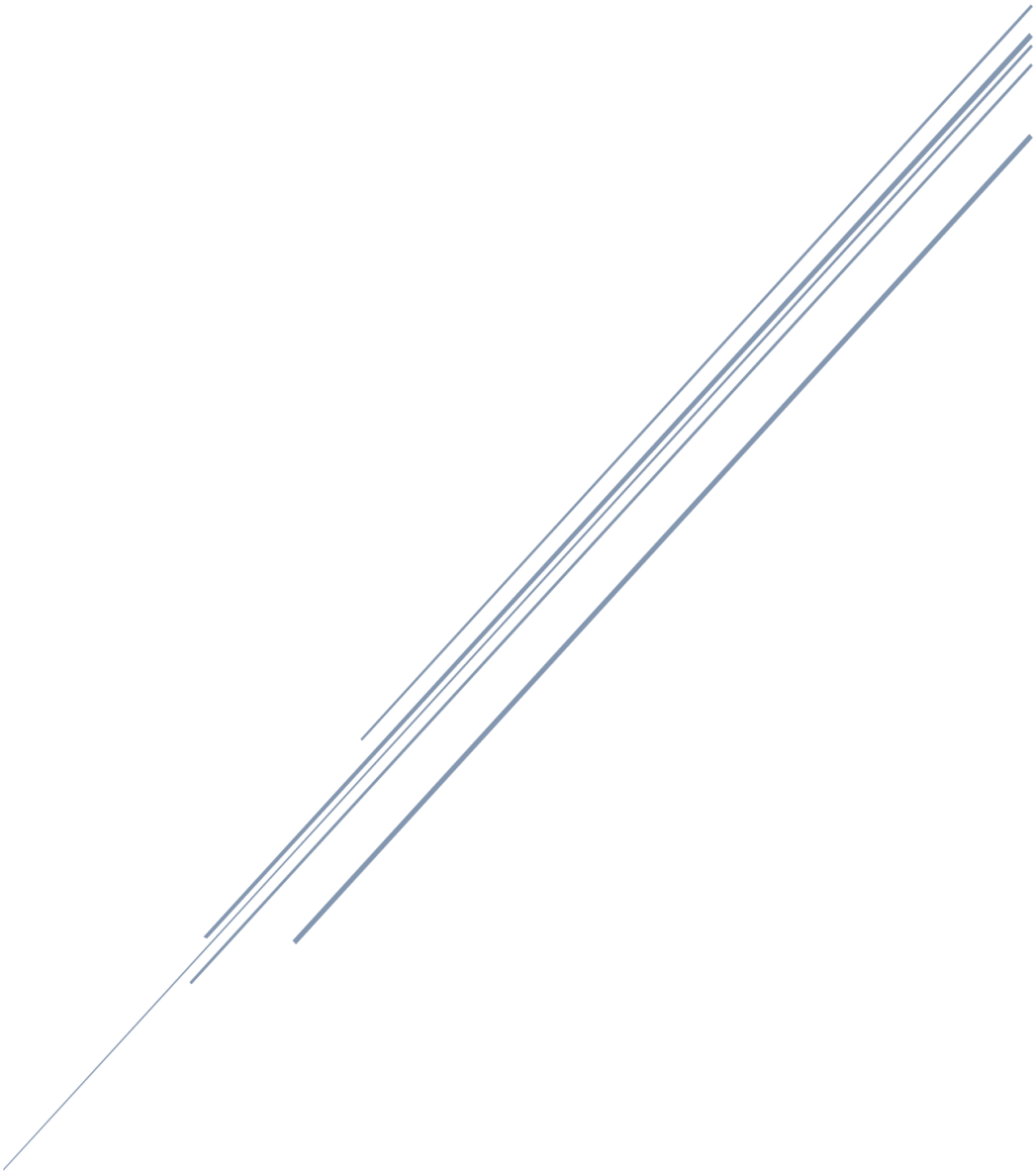
INTERNSHIP DAILY REPORT

Day 5 – Cybersecurity Basics (Offense + Defense)

CyberShelter Cybersecurity Internship | Week 1



Mohamed Shameem PA

Associate – Information Security (Intern)

13 November 2026

1. Objective

The objective of Day 5 was to learn the basics of cybersecurity from both the offense (attacker) and defense side, covering threats, malware, authentication, encryption and firewalls. The practical covered hashing, a password-cracking demo on my own test hash, firewall basics with ufw and iptables, and checking security tools. The hands-on task was to hash a password, crack a weak password using a demo wordlist, and explain the difference between hashing and encryption.

2. Lab Environment

- **Host Machine:** Windows PC
- **Virtualisation Software:** Oracle VirtualBox
- **Guest OS:** Kali Linux (64-bit), VM "Kali Lab"
- **Tools Used:** md5sum, sha256sum, John the Ripper, ufw, iptables, hydra, openssl
- **Note:** The password-cracking demo used only a hash I created myself on my own VM. No real account or service was targeted.

3. Topics Covered

3.1 Threats

A threat is anything that can harm a system or data. Common threats include malware, phishing, social engineering, denial-of-service (DoS) attacks and insider misuse. Attackers look for weaknesses, called **vulnerabilities**, to exploit.

3.2 Malware

Malware is malicious software made to damage a system or gain access to it. Knowing the main types helps in detecting and stopping them.

- **Virus:** attaches to a file and spreads when the file is run.
- **Worm:** spreads by itself across a network with no user action.
- **Trojan:** hides inside a normal-looking program.
- **Ransomware:** encrypts the victim's files and demands payment.
- **Spyware:** secretly collects information from the system.

3.3 Authentication

Authentication is how a system proves who a user is. It is based on three factors, and using more than one is much stronger.

- **Something you know:** a password or PIN.
- **Something you have:** a phone or security key.
- **Something you are:** a fingerprint or face.
- **MFA (Multi-Factor Authentication):** uses two or more of these factors, which is far safer than a password alone.

3.4 Encryption

Encryption scrambles data so that only someone with the correct key can read it. It protects data both when it is stored and when it is sent over a network.

- **Symmetric:** one key is used to both lock and unlock the data, for example **AES**. It is fast.
- **Asymmetric:** a **public key** locks the data and a **private key** unlocks it, for example **RSA**. This is used in HTTPS.

3.5 Firewalls

A firewall controls which network traffic is allowed in or out of a system, based on a set of rules. It blocks unwanted connections and is a basic but important defence.

- **ufw (Uncomplicated Firewall):** a simple, easy-to-use firewall for everyday rules.
- **iptables:** the advanced, low-level firewall that ufw is built on top of.

4. Tasks Performed

All commands were run in the Kali Linux terminal as the normal user shameem, inside a dedicated day5 folder.

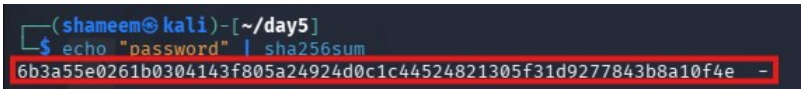
4.1 Hashing

4.1.1 MD5 and SHA-256 Hashing

- **Command Executed:** `echo "password" | md5sum and sha256sum`
- **Purpose:** Turns a password into a fixed-length hash using two different algorithms.
- **Observation:** The word "password" produced a short **MD5** hash and a longer **SHA-256** hash. SHA-256 is longer and considered more secure than MD5.



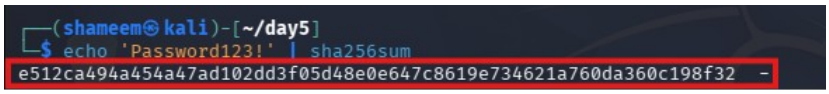
MD5 hash of the password.



SHA-256 hash of the password.

4.1.2 Hashing a Strong Password

- **Command Executed:** `echo 'Password123!' | sha256sum`
- **Purpose:** Shows that even a small change in the input gives a completely different hash.
- **Observation:** The strong password produced a totally different SHA-256 hash. The same input always gives the same hash, but any change produces a completely different one.

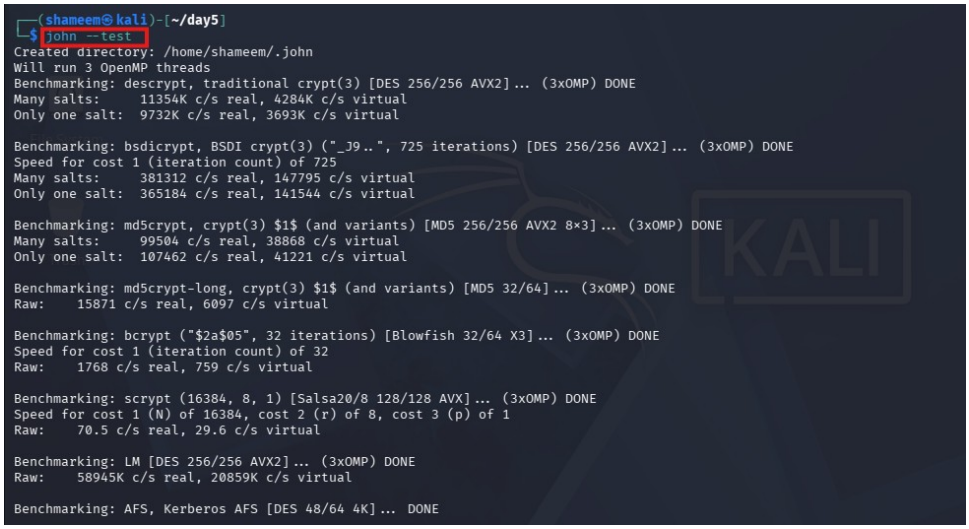


SHA-256 hash of a strong password.

4.2 Password Cracking Demo

4.2.1 John the Ripper Benchmark

- **Command Executed:** `john --test`
- **Purpose:** Runs John's built-in self-test to check the tool works and measure its speed.
- **Observation:** John benchmarked several hash types (descrypt, md5crypt, bcrypt, SHA and others) and showed how many hashes per second the machine can test. This is why weak passwords are cracked so quickly.

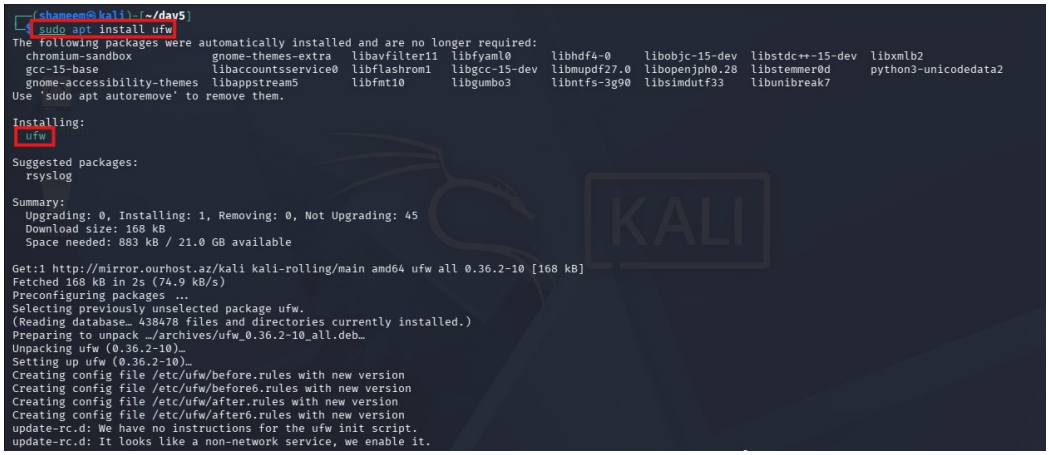


John the Ripper benchmark.

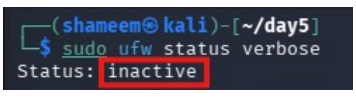
4.3 Firewall Basics

4.3.1 Installing and Checking UFW

- **Command Executed:** `sudo ufw status verbose`
- **Purpose:** Checks whether the ufw firewall is active.
- **Observation:** ufw was installed and its status was first shown as **inactive**, which means no firewall rules were being applied yet.



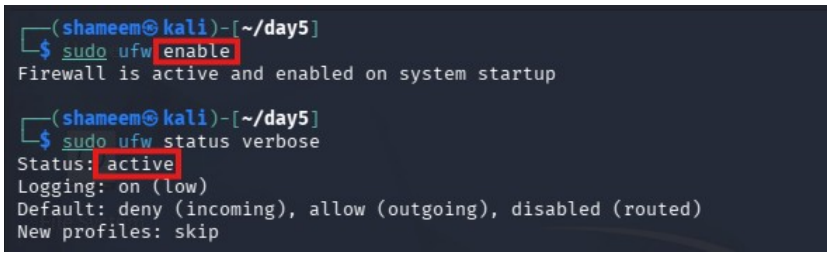
Installing ufw.



ufw status showing inactive.

4.3.2 Enabling the Firewall

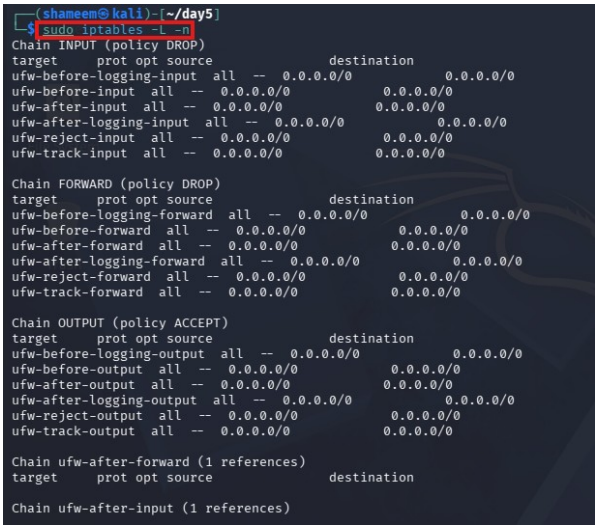
- **Command Executed:** `sudo ufw enable`
- **Purpose:** Turns the firewall on so its rules start protecting the system.
- **Observation:** After enabling, ufw became **active** and set to start on boot. The default policy is **deny incoming** and **allow outgoing**, which blocks unwanted connections while still allowing normal internet use.



Enabling ufw.

4.3.3 Viewing iptables Rules

- **Command Executed:** `sudo iptables -L -n`
- **Purpose:** Shows the low-level firewall rules that ufw created.
- **Observation:** The output shows the **INPUT**, **FORWARD** and **OUTPUT** chains, both with a default policy of **DROP** for incoming and forwarded traffic, and several ufw rules handling logging and tracking.

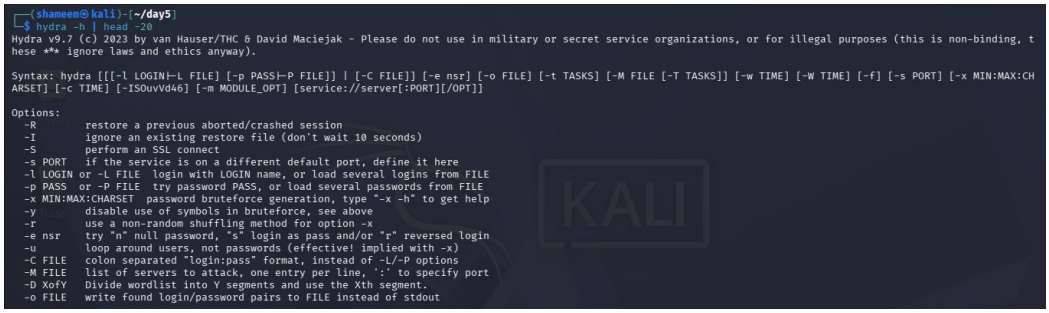


iptables rules created by ufw.

4.4 Security Tool Check

4.4.1 Hydra Help

- **Command Executed:** `hydra -h`
- **Purpose:** Shows the help text of hydra to confirm the tool is installed.
- **Observation:** Hydra 9.7 is installed. The help screen lists its options and clearly states it must **not** be used for illegal purposes. Only the help text was viewed; hydra was not run against any target.

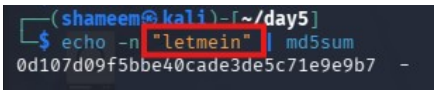


Hydra help screen.

4.5 Hands-on Task: Crack a Weak Password

4.5.1 Hash the Demo Password

- **Command Executed:** `echo -n "letmein" | md5sum`
- **Purpose:** Creates a demo MD5 hash of a weak password to use as the cracking target.
- **Observation:** The word "letmein" was hashed to **0d107d09f5bbe40cade3de5c71e9e9b7** and saved into a file called demohash.txt. The `-n` option was used so no extra newline was added.



Hashing the demo password.



Saving the hash to demohash.txt.



The demo hash file.

4.5.2 Create the Wordlist

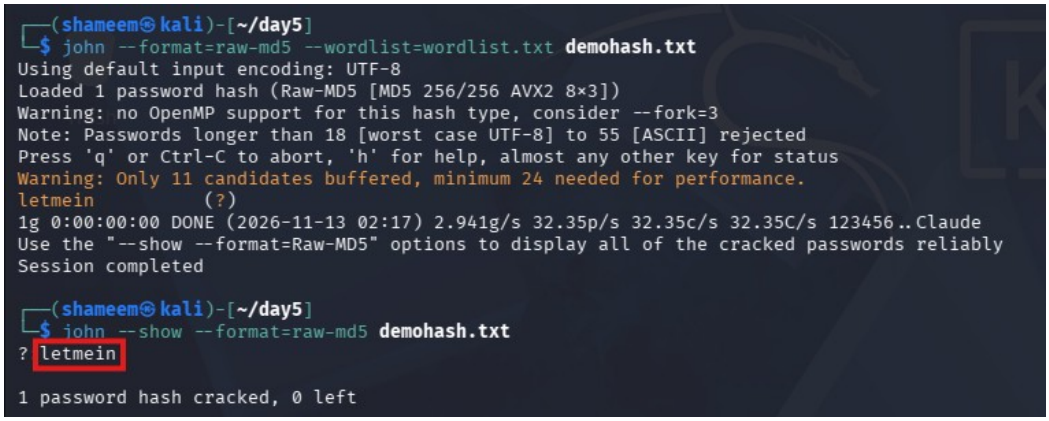
- **Command Executed:** `nano wordlist.txt`
- **Purpose:** Creates a small list of common weak passwords for John to try.
- **Observation:** A wordlist was created with common weak passwords, including **letmein**. These are the guesses John will test against the hash.



Creating the demo wordlist.

4.5.3 Crack the Hash

- **Command Executed:** `john --format=raw-md5 --wordlist=wordlist.txt demohash.txt`
- **Purpose:** Tries each word in the wordlist against the hash until one matches.
- **Observation:** John cracked the hash almost instantly and found the password **letmein**. `john --show` confirmed **"1 password hash cracked, 0 left"**. John does not reverse the hash; it hashes each word in the list and compares, which is why only weak, guessable passwords are found this way.



John cracking the weak password.

5. Results

5.1 Hands-on Task Findings

Item	Result
Password hashed	"password" and "letmein" using MD5 and SHA-256
Weak password cracked	letmein (MD5 0d107d09...e9b7), cracked instantly
Why it was cracked	It was a common password present in the wordlist
Firewall (ufw)	Changed from inactive to active, default deny incoming
Tools confirmed	John the Ripper and hydra installed and working

5.2 Difference between Hashing and Encryption

Feature	Hashing	Encryption
Direction	One-way (cannot be reversed)	Two-way (reversed with a key)
Key	No key used	Uses a key
Purpose	Store passwords, check integrity	Protect data that must be read again
Example	MD5, SHA-256	AES, RSA (HTTPS)

In the practical, John could only crack the weak password because **hashing is one-way**, so it had to guess words and hash each one. A strong password that is not in the wordlist cannot be cracked this way. This is why strong, unique passwords and **salting** are important.

6. Key Learnings

- Learned the main cybersecurity threats and types of malware, and how authentication and MFA protect accounts.
- Understood the difference between symmetric and asymmetric encryption and where each is used.
- Created password hashes with `md5sum` and `sha256sum` and saw how the same input always gives the same hash.
- Did a safe, self-contained password-cracking demo and saw why weak passwords fail and strong ones do not.
- Learned to check and enable the `ufw` firewall and view the underlying `iptables` rules.

7. Challenges Faced

- **Hash did not match at first:** the first demo hash was made with `echo "letmein"` which adds a hidden newline, so John could not match it. Using `echo -n "letmein"` removed the newline and the crack then worked.
- **History expansion error:** typing a password with `!` in double quotes caused a `dquote>` prompt. Using single quotes fixed it, because single quotes stop the shell from treating `!` specially.

8. Conclusion & Next Steps

Day 5 covered the basics of cybersecurity from both the attacker and defender side. The hashing and cracking demo made it clear why hashing is one-way and why weak passwords are dangerous, while the firewall tasks showed a simple but important defence. **Next Steps:** Day 6 will focus on reconnaissance and information gathering on authorised targets.